TOPIC THE LAB

Write history worth reading.

A commit is a saved snapshot with a note on it. Learn to write notes your future self will thank you for.

01

SEE IT SETUP




```
$ bash setup.sh
practice repo ready
```

builds six messy commits for you to read

DO IT TASK 1

1. **Check your version**

```
$ git --version
```

2.28 or newer is fine

2. **Say who you are**

```
$ git config --global user.name "Ada Example"
```

then user.email, the same way

3. **Pick an editor**

```
$ git config --global core.editor nano
```

or "code --wait" for VS Code

WATCH OUT !

The vim trap

No -m on git commit can open vim. To escape: Esc, :q!, Enter. Then set an editor.

```
Esc   :q!   Enter
```

TASK 2 READ

```
$ git log --oneline
```

Read the history.
Add --stat for files.

```
> 3f9a2c1 stuff
```

TASK 3 IGNORE

```
$ git check-ignore -v .env
```

Keep junk out.
Then commit .gitignore.

```
> .gitignore:1:.env
```

TASK 4 SPLIT

```
$ git add -p
```

Two changes, two commits.
s splits a big hunk.

```
> Stage this hunk [y,n,s]?
```

CHECK Q1

How do you leave vim without saving?

A:

CHECK Q2

What does --global change?

A:

COMPARE

✗ no identity set

✓ name and email set

COMPARE

✗ git picks vim

✓ you pick the editor

- DROP

starting without your setup

+ LEARN

set it once, then just commit

NEEDS

Git 2.28 or newer, a terminal and any text editor. On Windows, use Git Bash. Nothing is pushed anywhere in this lab. Also run: git config --global init.defaultBranch main

github.com/reflog-labs/lab-01-first-commit

TOPIC HOW GIT THINKS

A commit is a snapshot.

Git saves your files as they are right now, not a list of edits. Changes move through three places.

01

SEE IT THREE PLACES




```
$ git status -sb
M  menu.md
 M notes.md
```

left letter: staged. right letter: not yet

DO IT THE LOOP

1. **Stage by hunk**

```
$ git add -p
```

y stages, n skips, s splits a hunk

2. **Review what is staged**

```
$ git diff --staged
```

read it like a reviewer would

3. **Commit it**

```
$ git commit -m "Fix the Expresso typo"
```

no -m opens your editor

WATCH OUT !

The add-all trap

git add . stages everything, even .env and node_modules. Name files, or use -p.

```
$ git add menu.md
```

READ LOG

```
$ git log --oneline
```

One line per commit.
Short hash, then subject.

> 3f9a2c1 stuff

READ SHOW

```
$ git show 3f9a2c1
```

Full change for one hash.
Take it from the log.

> + Oat milk, 0.60

READ SEARCH

```
$ git log -S"oat"
```

Commits that add or drop the text oat.

> 8d2f0ab Add oat milk

CHECK Q3

What does the p in add -p mean?

A:

CHECK Q4

Which command shows staged work?

A:

COMPARE

✗ git add .

✓ git add -p

COMPARE

✗ commit, then hope

✓ diff --staged first

- DROP

committing without a look

+ LEARN

status, diff --staged, then commit

SNAPSHOT

Git stores a snapshot of your files at each commit and works out the difference by comparing two snapshots. That is why a small, focused commit is easy to read, review and undo.

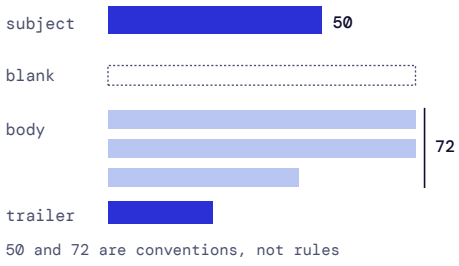
TOPIC MESSAGE CRAFT

Anatomy of a message.

A subject says what changed. A body says why. Trailers link issues and credit people.

01

SEE IT THE SHAPE



50 and 72 are conventions, not rules

DO IT WRITE IT

1. **Start with a verb**

```
Add oat milk to the menu
```

finish: "If applied, this commit will ..."

2. **Keep the subject short**

```
Fix the Expresso typo
```

about 50 characters, no full stop

3. **Explain why in the body**

```
$ git commit
```

opens your editor: subject, blank line, body

WATCH OUT !

WIP stays local

wip is fine on your machine while you work. Clean it up before anyone else sees it.

```
$ git commit -m "wip"
```

LEVEL UP OPTIONAL

```
$ git commit -s
```

Adds Signed-off-by.
Some projects ask for it.

› Signed-off-by: Ada ...

LEVEL UP OPTIONAL

```
$ git config commit.template
```

Pre-fills your editor with your own shape.

› ~/.gitmessage

LEVEL UP OPTIONAL

```
feat: add oat milk
```

Conventional Commits: type, colon, subject.

› feat: fix: docs: chore:

REWRITE

✗ stuff

✓ Add oat milk item

REWRITE

✗ fix

✓ Correct latte price

REWRITE

✗ final_v2

✓ Split hours section

REWRITE

✗ Update menu.md

✓ Fix Expresso typo

- DROP

wip, stuff and final_v2

+ LEARN

a subject a stranger can scan

READ

Chris Beams: How to Write a Git Commit Message (cbea.ms/git-commit)
Conventional Commits specification (conventionalcommits.org)
Pro Git, the free book (git-scm.com/book)

TOPIC HABITS

Habits that stick.

Look before you stage, keep junk out, and never commit a secret. Small habits make clean history.

01

SEE IT TASK 3




```
$ git check-ignore -v .env
.gitignore:1:.env   .env
```

if it prints a line, git ignores the file

DO IT SEARCH

1. **Search messages**

```
$ git log --grep oat --oneline
```

finds commits whose message says oat

2. **Search changes**

```
$ git log -S"oat" --oneline
```

finds commits that add or drop oat

3. **Read one in full**

```
$ git show 8d2f0ab
```

message plus the diff

WATCH OUT !

Leaked is leaked

Deleting a key in a new commit leaves it in history. Revoke it and make a new one.

```
$ git log -S"API_KEY"
```

TIP DIFF

```
$ git commit -v
```

Shows the diff while you write the message.

> diff --git a/menu.md

TIP PRIVACY

```
$ git config user.email
```

Use your GitHub noreply address in public repos.

> ID+name@users.noreply...

TIP STYLE

```
$ git log --oneline -5
```

Read the log first. Match the project style.

> 8d2f0ab Add oat milk

BEFORE COMMIT

- [ ] **Ran git status**

what changed?

BEFORE COMMIT

- [ ] **Read the staged diff**

nothing surprising

BEFORE COMMIT

- [ ] **No keys or .env files**

use .gitignore

BEFORE COMMIT

- [ ] **One idea per commit**

or split with -p

- DROP

committing secrets and junk

+ LEARN

.gitignore first, then commit

ANSWERS

Q1 Esc, :q!, Enter. Q2 Every repo for your user.
Q3 Patch, so you stage piece by piece. Q4 git diff --staged.
Next up: Lab 02 teaches branches, so you can break things safely.